Portfolio Project: Machine Learning with Python - Telecom Customer Classification with K-Nearest Neighbors

Data Science Portfolio — IBM Professional Certificate Projects | Module 3: Building Supervised Learning Models

👩‍💻 Executive Summary

This notebook documents the implementation of a KNN learning model to classify new customers for a telecommunications company. The project shows how to find the best possible value for k in order to obtain the model which can predict the class of a new customer with more accuracy.

Original lab framework provided by IBM Skills Network. Technical implementation, statistical analysis, and visualizations performed by Roberto Castro/rcastro-ai

Lab: K-Nearest Neighbors Classifier

Estimated time needed: **25** minutes

Objectives

After completing this lab you will be able to:

- Use K-Nearest neighbors to classify data
- Apply KNN classifier on a real world data set

In this lab you will load a customer data set, fit the data, and use K-Nearest Neighbors to predict a data point.

Import the libraries

First, to make sure that all required libraries are available, run the cell below.

In [ ]:
```
# Installing needed libraries

!pip install numpy==2.2.0
!pip install pandas==2.2.3
!pip install scikit-learn==1.6.0
!pip install matplotlib==3.9.3
!pip install seaborn==0.13.2
```

Now, let's load required libraries.

In [2]:
```
# Importing libraries

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
%matplotlib inline
```

About the data set

Imagine a telecommunications provider has segmented its customer base by service usage patterns, categorizing the customers into four groups. If demographic data can be used to predict group membership, the company can customize offers for individual prospective customers. It is a classification problem. That is, given the dataset, with predefined labels, we need to build a model to be used to predict class of a new or unknown case.

The example focuses on using demographic data, such as region, age, and marital status, to predict usage patterns.

The target field, called **custcat**, has four possible service categories that correspond to the four customer groups, as follows:

1. Basic Service
2. E-Service
3. Plus Service
4. Total Service

Our objective is to build a classifier to predict the service category for unknown cases. We will use a specific type of classification called K-nearest neighbors.

Load Data

Let's read the data using pandas library and print the first five rows.

In [3]:
```
# Loading the data into a dataframe

df = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-storage.appdomain.clou
df.head()
```

Out[3]:

	region	tenure	age	marital	address	income	ed	employ	retire	gender	reside	cus
0	2	13	44	1	9	64.0	4	5	0.0	0	2	
1	3	11	33	1	7	136.0	5	5	0.0	0	6	
2	3	68	52	1	24	116.0	1	29	0.0	1	2	
3	2	33	33	0	12	33.0	2	0	0.0	1	1	
4	2	23	30	1	9	30.0	1	2	0.0	0	4	

In [4]:
```
# Saving a local copy of the original data

#df.to_csv('tele_customer.csv')
```

Data Visualization and Analysis

Let us first look at the class-wise distribution of the data set.

In [5]:
```
# Showing the distribution of the class

df['custcat'].value_counts()
```

Out[5]:
```
custcat
3    281
1    266
4    236
2    217
Name: count, dtype: int64
```

Hence, we can say that we have records of 281 customers who opt for Plus Services, 266 for Basic-services, 236 for Total Services, and 217 for E-Services. It can thus be seen that the data set is mostly balanced between the different classes and requires no special means of accounting for class bias.

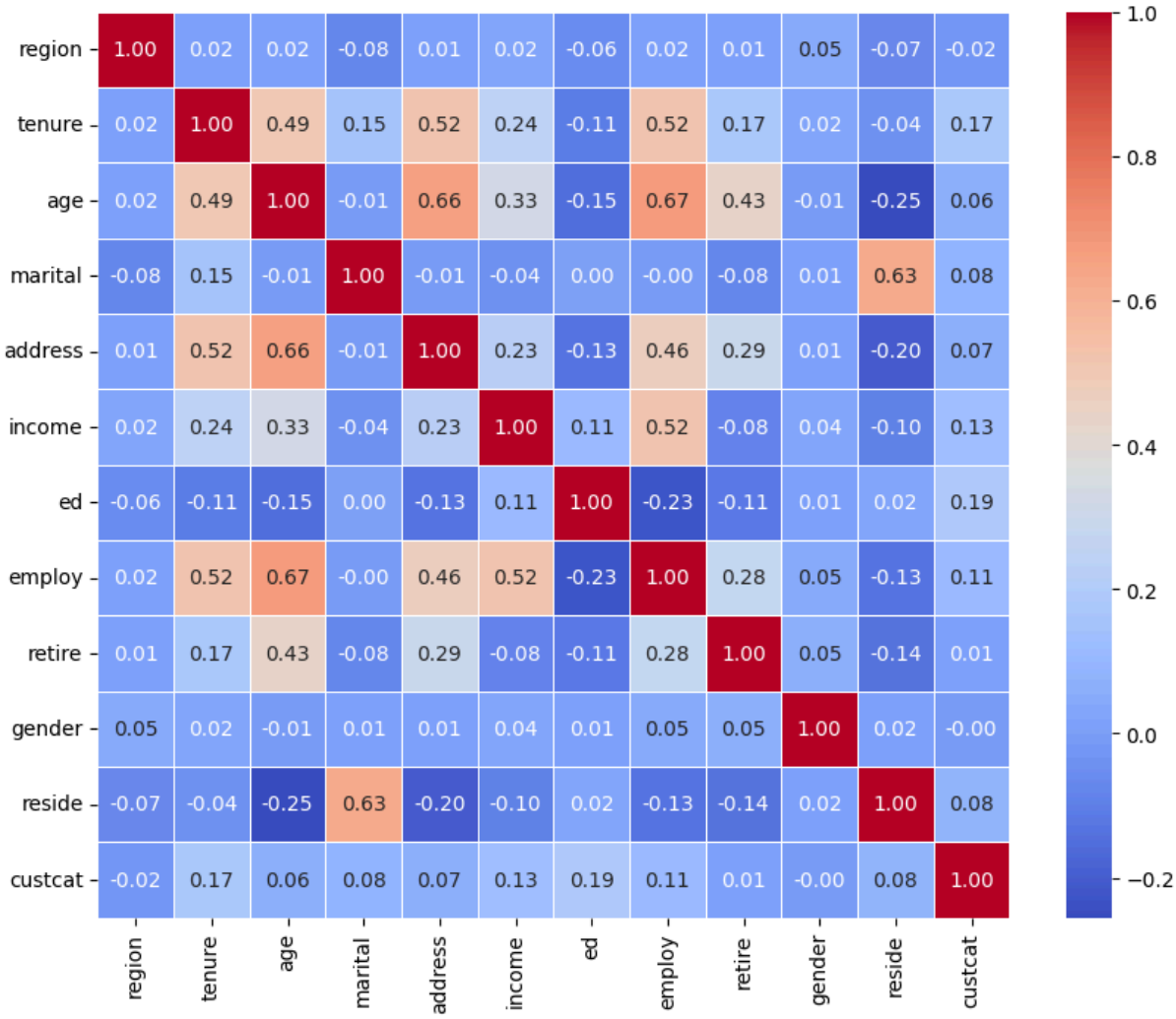
We can also visualize the correlation map of the data set to determine how the different features are related to each other.

In [6]:
```
# Showing the correlation of the features

correlation_matrix = df.corr()
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=
```

Out[6]: <Axes: >



As is visible from the correlation map, some features have beeter correlation among them than others, basically indicating the depth of relationship between the two features. What is of interest to us is the correlation of the target feature, i.e. `custcat` with all the other features. This will help us identify which features should be focussed on for modeling and which ones can be ignored.

The following code snippet will give us a list of features sorted in the descending order of their absolute correlation values with respect to the target field.

In [7]:
```
# Listing the correlation of the features with the target variable sorted by correl

correlation_values = abs(df.corr()['custcat'].drop('custcat')).sort_values(ascendin
correlation_values
```

Out[7]:
```
ed         0.193864
tenure     0.166691
income     0.134525
employ     0.110011
marital    0.083836
reside     0.082022
address    0.067913
age        0.056909
region     0.023771
retire     0.008908
gender     0.004966
Name: custcat, dtype: float64
```

This shows us that the features `retire` and `gender` have the least effect on `custcat` while `ed` and `tenure` have the most effect.

Separate the input and target features

Now, we can separate the data into the input data set and the target data set.

In [8]:
```
# Separating the independent variables from the target variable

X = df.drop('custcat',axis=1)
y = df['custcat']
```

Normalize Data

Data normalization is important for the KNN model.

KNN makes predictions based on the distance between data points (samples), i.e. for a given test point, the algorithm finds the k-nearest neighbors by measuring the distance between the test point and other data points in the dataset. By normalizing / standardizing the data, you ensure that all features contribute equally to the distance calculation. Since normalization scales each feature to have zero mean and unit variance, it puts all features on the same scale (with no feature dominating due to its larger range).

This helps KNN make better decisions based on the actual relationships between features, not just on the magnitude of their values.

In [9]:
```
# Normalizing the data

X_norm = StandardScaler().fit_transform(X)
```

Train Test Split

Now, you should separate the training and the testing data. You can retain 20% of the data for testing purposes and use the rest for training. Assigning a random state ensures reproducibility of the results across multiple executions.

In [10]:
```
# Splitting the data into train and test datasets

X_train, X_test, y_train, y_test = train_test_split(X_norm, y, test_size=0.2, rando
```

KNN Classification

Once the data is in place, we can now execute the training of the model.

Training

Initially, you may start by using a small value as the value of k, say k = 4.

In [11]:
```
# Starting with a small value for k

k = 3

# Train Model and Predict
knn_classifier = KNeighborsClassifier(n_neighbors=k)
knn_model = knn_classifier.fit(X_train, y_train)
```

Predicting

Once the model is trained, we can now use this model to generate predictions for the test set.

In [12]:
```
# Generating a prediction

yhat = knn_model.predict(X_test)
```

Accuracy evaluation

In multilabel classification, **accuracy classification score** is a function that computes subset accuracy. This function is equal to the jaccard_score function. Essentially, it calculates how closely the actual labels and predicted labels are matched in the test set.

In [13]:
```
# Evaluating the model

print("Test set Accuracy: ", accuracy_score(y_test, yhat))
```

```
Test set Accuracy:  0.315
```

Exercise 1

Can you build the model again, but this time with k=6?

In [15]:
```
# Creating a model with k = 6
```

```
k_6 = 6

# Train Model and Predict
knn_classifier_6 = KNeighborsClassifier(n_neighbors=k_6)
knn_model_6 = knn_classifier_6.fit(X_train, y_train)

# Generating a prediction
yhat_6 = knn_model_6.predict(X_test)

# Evaluating the model
print("Test set Accuracy: ", accuracy_score(y_test, yhat_6))
```

```
Test set Accuracy:  0.31
```

▶ Click here for the solution

Choosing the correct value of k

K in KNN, is the number of nearest neighbors to examine. However, the choice of the value of 'k' clearly affects the model. Therefore, the appropriate choice of the value of the variable `k` becomes an important task. The general way of doing this is to train the model on a set of different values of k and noting the performance of the trained model on the testing set. The model with the best value of `accuracy_score` is the one with the ideal value of the parameter k.

Check the performance of the model for 10 values of k, ranging from 1-9. You can evaluate the accuracy along with the standard deviation of the accuracy as well to get a holistic picture of the model performance.

In [16]:
```
# Testing the model for different values of k to select the best value

Ks = 10
acc = np.zeros((Ks))
std_acc = np.zeros((Ks))
for n in range(1, Ks+1):
    # Train Model and Predict
    knn_model_n = KNeighborsClassifier(n_neighbors = n).fit(X_train,y_train)
    yhat = knn_model_n.predict(X_test)
    acc[n-1] = accuracy_score(y_test, yhat)
    std_acc[n-1] = np.std(yhat==y_test)/np.sqrt(yhat.shape[0])
```

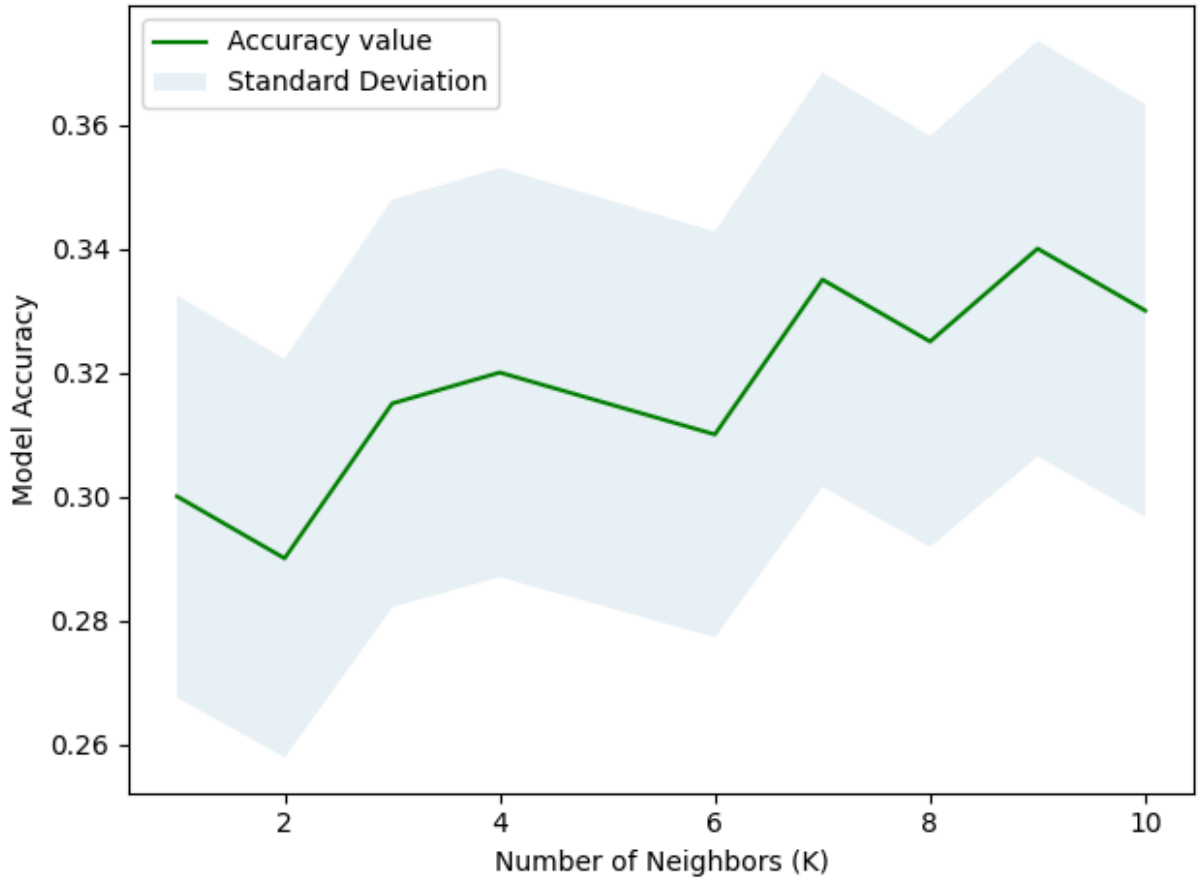
Plot the model accuracy for a different number of neighbors.

Now, you can plot the model accuracy and the standard deviation to identify the model with the most suited value of k.

In [17]:
```
# Plotting the model accuracy

plt.plot(range(1, Ks+1), acc, 'g')
plt.fill_between(range(1, Ks+1), acc - 1 * std_acc, acc + 1 * std_acc, alpha=0.10)
plt.legend(('Accuracy value', 'Standard Deviation'))
```

```
plt.ylabel('Model Accuracy')
plt.xlabel('Number of Neighbors (K)')
plt.tight_layout()
plt.show()
```



In [18]:
```
# Printing the best accuracy and its related k value

print( "The best accuracy was with", acc.max(), "with k =", acc.argmax()+1)
```

```
The best accuracy was with 0.34 with k = 9
```

However, since this graph is still rising, there can be a chance that the model will give a better performance with an even higher value of k.

Exercise 2

Run the training model for 30 values of k and then again for 100 values of k. Identify the value of k that best suits this data and the accuracy on the test set for this model.

Enter you answer here

In [19]:
```
# Running the model for 30 and 100 values of k

for Ks in [30, 100]:
    acc = np.zeros((Ks))
    std_acc = np.zeros((Ks))
    for n in range(1, Ks+1):
```

```
        # Train Model and Predict
        knn_model_n = KNeighborsClassifier(n_neighbors = n).fit(X_train,y_train)
        yhat = knn_model_n.predict(X_test)
        acc[n-1] = accuracy_score(y_test, yhat)
        std_acc[n-1] = np.std(yhat==y_test)/np.sqrt(yhat.shape[0])

    # Printing the best accuracy and its related k value
    print("For the test of k from 1 to ", Ks, ", the best accuracy was ", acc.max()
```

```
For the test of k from 1 to  30 , the best accuracy was  0.385 with k = 30
For the test of k from 1 to  100 , the best accuracy was  0.41 with k = 38
```

▶ Click here for answer

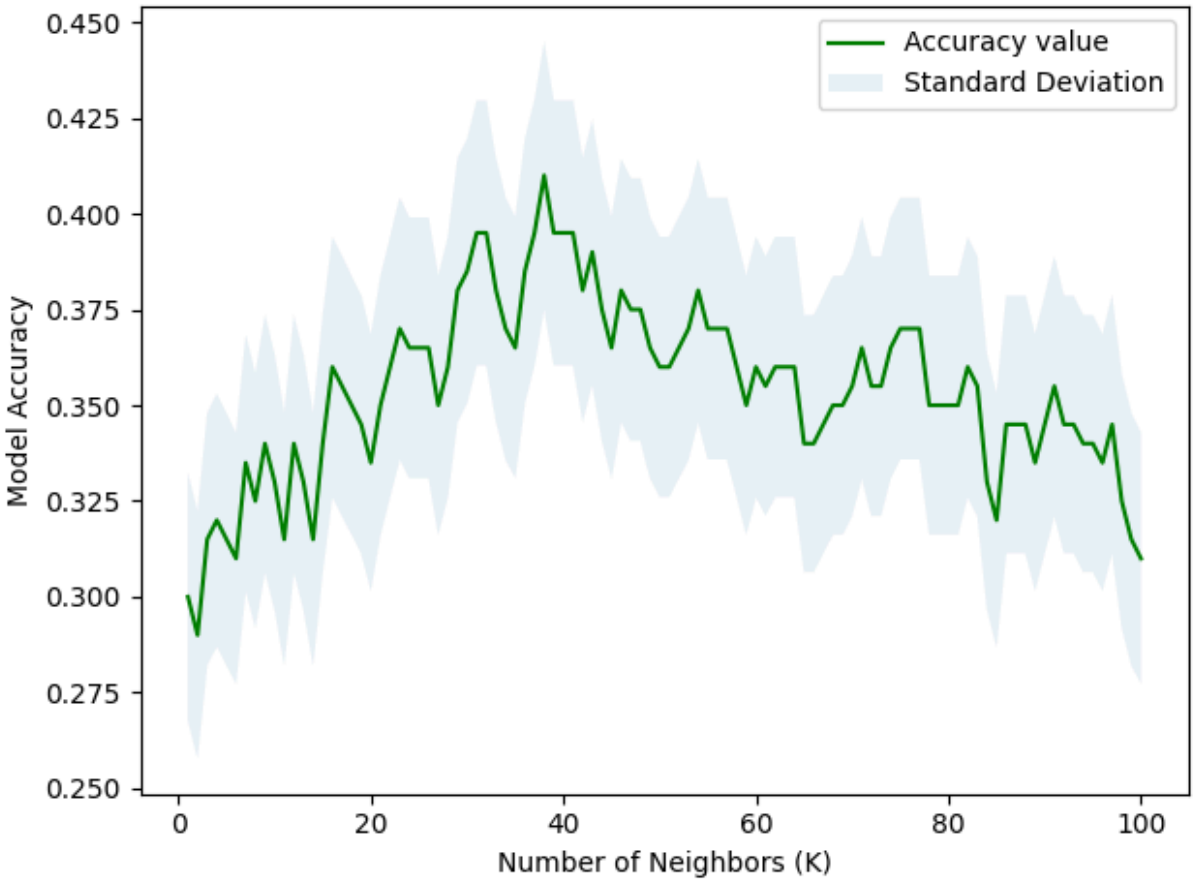
Exercise 3

Plot the variation of the accuracy score for the **training set** for 100 value of Ks.

In [20]:
```
# Plotting the variation of the accuracy for 100 values of k

for Ks in [100]:
    acc = np.zeros((Ks))
    std_acc = np.zeros((Ks))
    for n in range(1, Ks+1):
        # Train Model and Predict
        knn_model_n = KNeighborsClassifier(n_neighbors = n).fit(X_train,y_train)
        yhat = knn_model_n.predict(X_test)
        acc[n-1] = accuracy_score(y_test, yhat)
        std_acc[n-1] = np.std(yhat==y_test)/np.sqrt(yhat.shape[0])

# Plotting the model accuracy
plt.plot(range(1, Ks+1), acc, 'g')
plt.fill_between(range(1, Ks+1), acc - 1 * std_acc, acc + 1 * std_acc, alpha=0.10)
plt.legend(('Accuracy value', 'Standard Deviation'))
plt.ylabel('Model Accuracy')
plt.xlabel('Number of Neighbors (K)')
plt.tight_layout()
plt.show()
```



▶ Click here for the solution

Exercise 4

Can you justify why the model performance on training data is deteriorating with increase in the value of k?

Enter you answer here

Answer: As the value of k increases, the model starts to underfit, losing accuracy.

▶ Click here for the solution

Exercise 5

We can see that even the with the optimum values, the KNN model is not performing that well on the given data set. Can you think of the possible reasons for this?

Enter you answer here

Answer: My first thought would be the size of the dataset. If it can be increased, that will help with the model accuracy. Another reason could be that we need better features.

▶ Click here for the solution

Congratulations! You're ready to move on to your next lesson!

Author

Abhishek Gagneja

Other Contributors

Jeff Grossman

© IBM Corporation. All rights reserved.